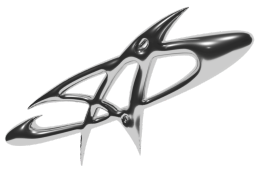
Week 03 · Attention Mechanism

모든 단어가 서로를 직접 바라본다면

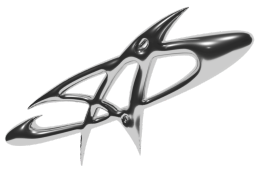
Query, Key, Value라는 세 가지 벡터로 Self-Attention의 직관을 잡고, Scaled Dot-Product Attention을 수식 그대로 손으로 구현합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이론 세션

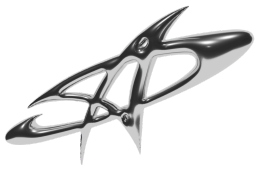
Self-Attention의 핵심, Query · Key · Value



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

RNN/LSTM은 정보를 순서대로 전달해야 했다.
오늘은 그 대신, 모든 단어 쌍을 직접 연결하는 방법을 만든다.

이 메커니즘의 이름이 Self-Attention이다.

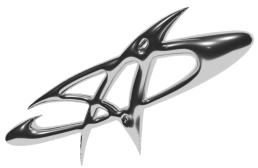


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Query / Key / Value — 도서관 비유

각 단어가 “질문”하고, “색인”에 답하고, “내용”을 건넨다

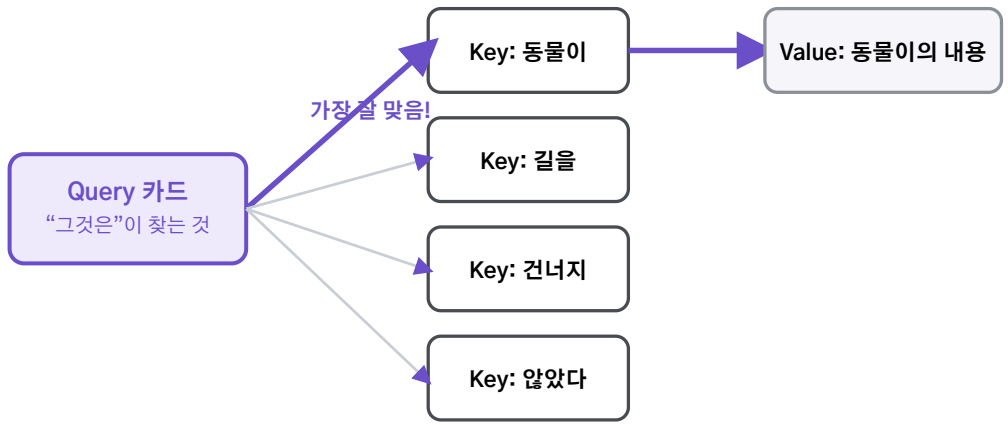
- **Query(질문)** — “나는 지금 어떤 정보가 필요한가?” (내가 찾는 것)
- **Key(색인)** — “나는 이런 정보를 갖고 있어” (책의 색인 카드)
- **Value(내용)** — “내가 실제로 줄 수 있는 내용” (책의 본문)
- 모든 단어는 **동시에** Query이면서 Key이자 Value가 된다 — 그래서 “Self”-Attention
- 내 Query를 모든 단어의 Key와 비교해서, 가장 관련 있는 단어의 Value를 많이 가져온다



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

도서관 비유를 그림으로

질문 카드 하나로, 모든 색인을 훑는다



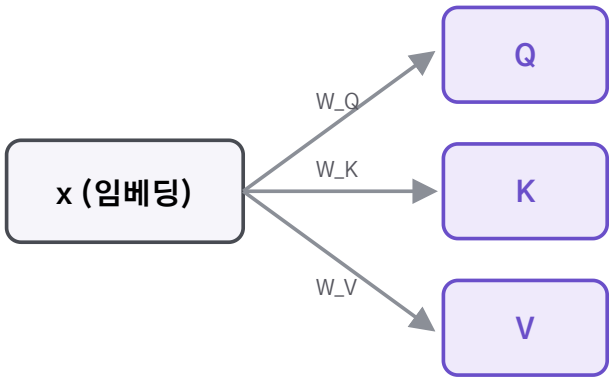
← 이 내용을 가장 많이 가져와
“그것은”의 새 표현을 만든다

Query 카드를 서가의 모든 Key 색인과 비교해 가장 잘 맞는 카드를 찾고, 그 카드에 연결된 Value(본문)를 가져오는 것 — 이것이 Attention의 직관입니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Q, K, V는 어디서 오는가

같은 입력 임베딩에, 서로 다른 3개의 선형변환

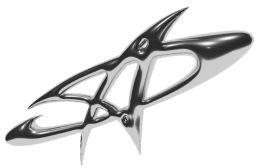


Q = xW_Q ← 이번 단어가 무엇을 찾는지

K = xW_K ← 이번 단어가 무엇을 갖고 있는지

V = xW_V ← 이번 단어가 실제로 건넬 내용

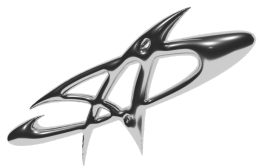
W_Q, W_K, W_V는 모두 학습되는 파라미터 — 학습이 진행되며 “무엇을 질문/색인/내용으로 쓸지”를 스스로 익힌다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

수식이 되기 전에, 먼저 4단계로 나눠보면

- **1. 유사도 계산** — 내 Query와 모든 단어의 Key를 내적(dot product)해서 “얼마나 관련 있는지” 점수화
- **2. 스케일링** — 점수를 √dk (Key 차원의 제곱근)로 나눠 값이 너무 커지지 않게 조정
- **3. Softmax** — 점수들을 합이 1인 확률 분포로 변환 (누구를 얼마나 볼지 비율)
- **4. 가중합** — 그 확률을 가중치 삼아 모든 단어의 Value를 더함 → 최종 출력



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

전체를 하나의 수식으로

Scaled Dot-Product Attention

방금 본 4단계가 정확히 이 한 줄의 수식입니다.

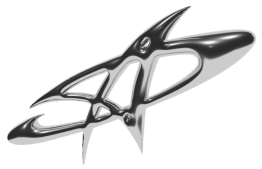
$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q,K,V — 각각 [문장 길이 × d_k] 크기의 행렬 (문장 전체를 한 번에 처리)

QK^T — 모든 단어 쌍의 내적 — [문장 길이 × 문장 길이] 크기의 "유사도 행렬"

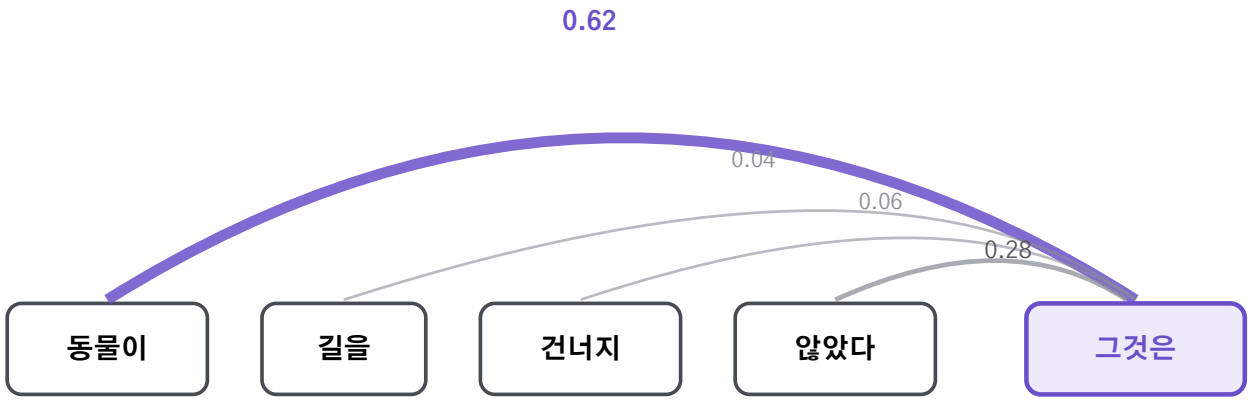
$\sqrt{d_k}$ — 스케일링 상수 (다음 슬라이드에서 이유 설명)

softmax(·)V — 각 행(=각 단어)마다 확률 가중합으로 새 벡터를 만듦



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

예시: “그것은”이 어떤 단어를 많이 보는가



학습이 잘 되면 “그것은”의 attention 가중치가 “동물이”에 가장 크게 실린다 — 대명사가 가리키는 대상을 스스로 찾아낸 것.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

왜 하필 √dk로 나누는가

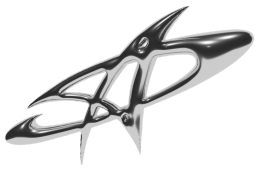
스케일링(Scaling)이 필요한 이유

d_k(Key 차원)가 커질수록 QK^T의 내적값 자체가 커지는 경향이 있습니다.

$$\mathrm{Var}(q \cdot k) \approx d_k \rightarrow \frac{q \cdot k}{\sqrt{d_k}} \text{ 로 나누면 분산이 다시 1 근처로}$$

문 제 — 내적값이 너무 크면 softmax의 값이 한쪽으로 극단적으로 쏠려, 기울기가 거의 0이 됨

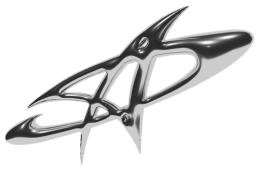
해 결 — √d_k로 나눠 값의 크기를 일정하게 유지 → softmax가 더 안정적으로 학습됨



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

코딩 세션

Scaled Dot-Product Attention 처음부터 직접 구현하기



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

문장 임베딩과 Q, K, V 만들기

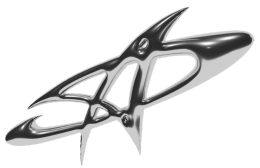
01_qkv.py

```
import torch, torch.nn as nn, math

seq_len, d_model, d_k = 5, 16, 16
x = torch.randn(1, seq_len, d_model)   # [batch, seq_len, d_model]

W_q = nn.Linear(d_model, d_k, bias=False)
W_k = nn.Linear(d_model, d_k, bias=False)
W_v = nn.Linear(d_model, d_k, bias=False)

Q, K, V = W_q(x), W_k(x), W_v(x)
print(Q.shape, K.shape, V.shape)  # [1, 5, 16] 각각
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

2단계 · 유사도 + 스케일링

$QK^T / \sqrt{d_k}$

02_scores.py

```
scores = Q @ K.transpose(-2, -1)     # [1, 5, 5] — 모든 단어 쌍의 내적
scores = scores / math.sqrt(d_k)      # 스케일링

print(scores.shape)  # [1, 5, 5]
# scores[0, i, j] = i번째 단어의 Query와 j번째 단어의 Key의 유사도
```



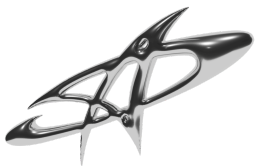
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

확률로 바꾸고, Value를 가중합

03_attention_output.py

```
weights = torch.softmax(scores, dim=-1)   # 각 행의 합이 1
output = weights @ V                        # [1, 5, 16]

print(weights[0, 0])  # 0번째 단어가 다른 단어들을 보는 비율
print(output.shape)   # 입력과 같은 [1, 5, 16] — 문맥이 반영된 새 표현
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

하나의 함수로 정리

scaled_dot_product_attention()

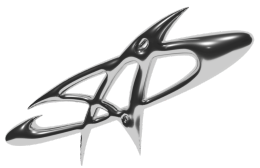
04_sdpa.py

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = (Q @ K.transpose(-2, -1)) / math.sqrt(d_k)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))

    weights = torch.softmax(scores, dim=-1)
    output = weights @ V
    return output, weights
```

mask 파라미터는 지금은 그냥 자리만 잡아둡니다 — 6주차 Decoder에서 “미래 단어를 못 보게 가리는 용도”로 그대로 재사용합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

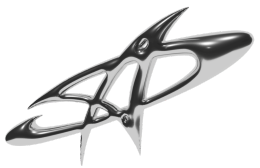
Attention score 시각화

weights 행렬을 히트맵으로 눈으로 확인

- matplotlib.pyplot.imshow로 [seq_len, seq_len] 가중치 행렬을 바로 그릴 수 있음
- 색이 밝을수록(값이 클수록) “그 단어를 많이 본다”는 의미

```
import matplotlib.pyplot as plt

w = weights[0].detach().numpy()   # [seq_len, seq_len]
plt.imshow(w, cmap="Purples")
plt.xticks(range(seq_len), tokens, rotation=45)
plt.yticks(range(seq_len), tokens)
plt.colorbar()
plt.title("Self-Attention weights")
plt.show()
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

3주차 정리

오늘 배운 것 & 다음 주 예고

오늘 배운 것

→Q, K, V = 질문 / 색인 / 내용

→Attention(Q,K,V) = softmax(QKᵀ/√dk)V

→√dk 스케일링은 softmax를 안정화하기 위함

→전체를 하나의 함수로 직접 구현함

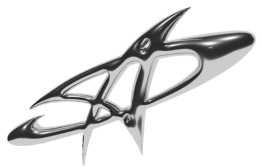
4주차 예고

→하나의 Attention만으로는 부족한 이유

→Multi-Head Attention — 여러 관점으로 동시에 보기

→Positional Encoding — 순서 정보를 어떻게 넣을까

→코딩: MultiHeadAttention 클래스 직접 구현



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지